Question	Answer
A neural network starts with random weights. Why don't the weights become more random after seeing errors?	The weights start random, but the gradient is not random: it tells each weight which direction would reduce the current loss. So gradient descent updates weights systematically in the opposite direction of the error gradient, gradually making them less random and more useful for the task. Prof. will teach that through backprpagation
how backpropagation helps gradient decent	Backpropagation is what calculates the gradients. It moves the error backward through the network and finds how much each weight contributed to it. Gradient descent then uses those gradients to decide how to update each weight. So backprop supplies the direction, gradient descent takes the step. Say true value is 10, model predicts 8, so error is 2. Backprop works backward and finds weight w1 caused most of it, giving gradients like w1: 4, w2: 1. Gradient descent then updates each weight against its gradient, so w1 changes more than w2. Next prediction moves closer to 10.
when we say model is hallucinating how will that relate to learning value.	Hallucination usually indicates a low learning value for that output, because the model is generating information that is not supported by the training/context evidence. However, it can still reveal what the model has incorrectly learned or generalized, which is useful for diagnosing and improving the model.
At some point, does the back propogation stop once the prediction becomes very accurate? Where does the true label come from?	Backpropagation does not stop just because one prediction becomes accurate, training usually stops after a fixed number of epochs or through early stopping when validation loss stops improving.
why MSE for Regression? is it dervied from probablity distribution?	Yes, it is. If you assume the errors follow a normal Gaussian distribution, then maximising the likelihood of your data works out to be exactly the same as minimising MSE. So MSE is not an arbitrary choice, it comes naturally from that assumption. It also helps that squaring keeps errors positive, punishes big mistakes more, and is smooth to differentiate for gradient descent.
Is binary Cross-Entropy is similar to log-loss?	yes. BCE is similar to log loss where the number of classes is 2
what is generalized linear regression?	Generalized linear models extend ordinary linear regression to targets that are not continuous or normally distributed. You still compute a weighted sum of inputs, but then pass it through a link function that maps it to the right range. For example, logistic regression is a GLM using a sigmoid link for 0/1 outputs, and Poisson regression uses a log link for count data.

How to make sure that Hallunication will be least in a model output	In general, hallucination can be reduced using better training data, stronger instruction tuning, verification with trusted sources, and prompting the model to abstain when evidence is insufficient. But this is not scope of this lecture today.
what is the main objective of Backpropagation in NN	Its main objective is to find out how much each weight in the network contributed to the final error. It computes the gradient of the loss with respect to every weight, working backward layer by layer using the chain rule. Those gradients are then used to update the weights and reduce the loss.
My understanding that the loss function measures how wrong the prediction is, while backpropagation calculates how each weight contributed to that error and in which direction the weights should be adjusted ?	Yes, it is correct. The loss function measures how wrong the prediction is, while backpropagation computes how much each weight contributed to that loss and the gradient direction, gradient descent then uses those gradients to update the weights.
Why do neural networks use a loss function like Binary Cross-Entropy instead of directly optimizing accuracy?	Because accuracy is not differentiable. It only changes in jumps when a prediction flips from wrong to right, so its gradient is zero almost everywhere and gradient descent gets no direction to move in. Cross-entropy is smooth. It reacts to how confident the prediction was, so even a small weight change gives a usable gradient.
Loss is calculated first. Backpropagation uses that loss to calculate the gradients needed to correct the weights, is that correct ?	Yes, exactly. First the loss is calculated from prediction versus true label, then backpropagation uses that loss to compute gradients for each weight.
what is the significance of taking log of the predicted value	The log punishes confident mistakes very heavily. If the true class gets probability 0.9 the loss is small, but if it gets 0.01 the loss shoots up, so the model is pushed hard to fix confident errors. It also turns products of probabilities into sums, which is easier and more stable to compute and differentiate.
as per my understanding, if I want to calculate the loss, ground truth is a mandatory parameter. is that correct?	Yes, for standard supervised learning, a ground-truth target is needed to calculate losses such as MSE or cross-entropy because the prediction is compared with the true value. However, not every learning setup requires human-provided labels, self-supervised and unsupervised methods can construct their own training targets
In ML also, gradient of loss wrt weight is evaluated and then weights are updated. How is the backpropagation in DL is different from ML?	The idea is the same, only the depth differs. In classical ML the model is usually one layer, so the gradient of loss with respect to a weight is a direct, single-step calculation. In deep learning there are many stacked layers, so backprop applies the chain rule repeatedly, passing the error from the output back through each layer to reach the early weights.

Why does Binary Cross-Entropy penalize confident wrong predictions so heavily?	Binary Cross-Entropy uses a logarithm, so if the model is very confident but wrong, the predicted probability of the true class becomes very close to 0 and −log(p) becomes very large. That is why a prediction like 0.05 for a true class of 1 gets a much bigger penalty than a less-confident mistake.
Can we use combination of all loss functions in adjusting weights?	Yes, multiple loss functions can be combined into one total objective, for example L_total = L1 + λL2, and backpropagation then updates weights using the gradient of that combined loss. But we should combine losses only when they represent meaningful objectives for the same task, not simply use all losses together.
In Binary Cross Entropy, are there examples other than Correct and Confident, Incorrect and Confident?	Yes, there are four cases in all. Besides confident and correct, and confident and wrong, you also get unsure and correct, and unsure and wrong. For a true label of 1, a prediction of 0.55 is unsure but correct and gives a moderate loss around 0.6. A prediction of 0.45 is unsure and wrong, giving a slightly higher loss around 0.8.
on the usecases perspective, how different is MSE and BCE?	From a use-case perspective, MSE is mainly used for regression, where the model predicts a continuous value such as house price or temperature, while BCE is mainly used for binary classification, where the model predicts the probability of one of two classes, such as spam/not spam or disease/no disease. MSE measures how far the predicted numeric value is from the true value, whereas BCE measures how well the predicted probability matches the correct class.
what is the difference between loss function and evaluation metrics? same functions are used.	The loss function is what the model trains on. It must be smooth and differentiable so gradients can flow. The evaluation metric is what you report to judge quality, and it only needs to be meaningful to humans.
Is there a validation Set for un-supervised learning?	Yes, but its role is different. Since there are no labels, you cannot check correctness, but you can still hold out data to tune choices like the number of clusters or the embedding size. You then judge it using unsupervised measures such as reconstruction error for autoencoders or silhouette score for clustering.
any link/pdf to see/read more example of Backpropagation and loss functions working in NN	you can go through the pre-reads provided
how we decide on the number of clusters in unsupervised learning?	- The elbow method plots the within-cluster error against the number of clusters and you pick the point where the curve bends. - The silhouette score measures how well separated the clusters are, and you pick the number giving the highest score. Domain knowledge matters too, since the clusters must make business sense.

how do you validate if the test data is adequate to validate your model or needs to be better ?	Check three things. First, is it large enough that the score is stable, not swinging when you resample it. Second, does it cover all classes and segments in similar proportion to real data. Third, is it truly unseen and collected the same way as production data. If the score changes a lot across different splits, your test set is too small.
Quick question.. Yday we learnt kfold validation. Will Kfold lead to data leakage into training ? as model will start seeing all of the data at one point?	No, standard k-fold cross-validation does not cause data leakage because in each fold the model is trained only on the training portion and evaluated solely on the held-out fold it has never seen, so even though the model sees the entire dataset across all folds combined, no single trained model instance is ever evaluated on data it was trained on; leakage only creeps in if you do preprocessing like scaling, feature selection, or encoding on the whole dataset before splitting into folds.
For a specific use case, how do you determine the correct or appropriate loss function to use? If I want to create my own custom loss functions, how can I go about determining the type of function needed?	Start from the output type. Continuous number means MSE or MAE, two classes means binary cross-entropy, many exclusive classes means categorical cross-entropy. For a custom loss, first write down what a bad prediction costs you in business terms. Then design a function that gives a low value for good predictions and a high value for bad ones, and make sure it is smooth and differentiable so gradients can flow. For example, if under-predicting demand costs more than over-predicting, weight the negative errors more heavily.
Could “Am I a robot?” CAPTCHA can be use the same neural-network concepts in e learning From Forward Pass to Prediction to Loss and Back propagation then weight update through Gradient Descent	Yes, that is a fair analogy. The image CAPTCHA is a labelled classification task, and Google has used those human labels as training data for image models.
question : how to decide which model will best suit for us as per input data ? could you please give one simple example and explain . if possible same for underfitting and overfitting as well.	Start from the data type and the output type. Tabular numbers with a continuous output means regression, images means CNN, text or sequences means RNN or transformer. Take house price prediction with 3 features. A straight line ignores the curve in the data, so both training and test error stay high, that is underfitting. A very high degree curve passes through every point, training error near zero but test error high, that is overfitting. A small curve or a shallow network fits the trend without chasing noise, and that is the good fit.

What does “model memorises training data” mean?	Model memorizes training data means the model learns the specific, idiosyncratic details including noise and random quirks of the individual training examples rather than the underlying general patterns, so it performs very well on that exact training data but fails to generalize and performs poorly on new, unseen data, which is a hallmark of overfitting.
how to handle unseen data or extrapolation?	Neural networks interpolate well but extrapolate poorly, since they only learn the range they were trained on. So the first fix is data, make sure your training range actually covers the values you will see later. Beyond that, keep the model simpler, add regularisation, and monitor inputs in production to flag when they fall outside the training range so you can retrain rather than trust the prediction.
which metric can help us to know if the model is overfitting?	There is no single metric specifically for overfitting. The key is to compare training and validation performance. If training loss keeps decreasing or training accuracy keeps increasing, while validation loss starts increasing or validation accuracy stops improving, the model is likely overfitting. So the most useful indicators are usually training vs. validation loss, along with metrics such as accuracy, F1-score, or RMSE depending on the task.
I have a question on model training post production deployment. a) Is the model training on going in the backend even when the model is deployed to prod env b) If so, how are improvements in the model with new training data propogated to production	Usually no: once a model is deployed, the production model is used for inference while any further training happens separately in an offline training pipeline. New data is collected, a new model version is trained and validated, and only after evaluation is that new version deployed to production through mechanisms such as model versioning, canary deployment, or A/B testing.
how is the derivative calculated. for taht we need to plot the weight against cost so do we need to run 1 epoch first then caculate backprop from 2nd epoch?	No plot is needed, and no waiting for a second epoch. The derivative is calculated using calculus, not by drawing the curve. Each operation in the network has a known derivative formula, and backprop chains them together. So it happens within a single forward pass. You do the forward pass, compute the loss, then immediately run backprop and update the weights. This repeats for every batch.
why we are doing square of bias?	In the bias-variance decomposition, bias is squared because the prediction error itself is squared, so the contribution from systematic error naturally appears as Bias². Squaring also makes negative and positive bias contribute positively to total error, since both represent error magnitude.
what is intuition behind bias^2 in Total Error = bias^2 + variance + IRN? => why not bias + variance + IRN?	we square the bias because bias is a signed error: positive and negative errors could cancel each other if we simply averaged them. Squaring makes the contribution always non-negative and also penalizes larger systematic errors more strongly. That is why prediction error is commonly decomposed into Bias² + Variance + IRN.

how are bias and variance calculated?	Bias and variance describe two different ways a model can make errors. Imagine training the same type of model many times on slightly different training datasets. $\text{Bias} = E[\hat{y}] - y$ Here, $\hat{y}$ is one model prediction, $E[\hat{y}]$ is the average prediction across many trained models, and y is the true value. So bias asks: “On average, how far is the model’s prediction from the truth?” $\text{Variance} = E[(\hat{y} - E[\hat{y}])^2]$ This measures how much the predictions change from one trained model to another. So variance asks: “If I retrain the same model on different data, how much do its predictions keep changing?” In short, high bias means the model is consistently wrong, while high variance means the model is unstable and changes too much depending on the training data.
where to use L1 and L2. and why not always use L2	Use L1 regularisation when you want sparsity, meaning some weights become exactly zero, which can help with feature selection and simpler models. Use L2 regularisation when you mainly want to shrink large weights smoothly while keeping most features; we do not always use L2 because sometimes sparsity is desirable, and different problems benefit from different regularisation behaviour.
When is normalization used	Use it whenever your features are on very different scales, say age in years and income in lakhs. Without it the large-scale feature dominates the gradients and training becomes slow or unstable. It is essential for neural networks, gradient descent, distance-based methods like KNN, and also before regularization, since L1 and L2 penalise all weights equally and unequal scales would distort that.

In What cases we use Lasso and Ridge ? How to determine what to use ?	Use Ridge (L2) when you believe most or all features are somewhat useful/relevant and you mainly want to shrink coefficients to reduce variance and handle multicollinearity, since it shrinks coefficients smoothly toward zero but never sets them exactly to zero. Use Lasso (L1) when you suspect many features are irrelevant or redundant and want automatic feature selection, since it can shrink some coefficients exactly to zero, effectively removing them from the model and producing a sparser, more interpretable solution. As a middle ground, Elastic Net (combining L1 and L2) is useful when you have many correlated features and want both shrinkage and selection, since pure Lasso tends to arbitrarily pick one feature among correlated groups while dropping the others. In practice, the best way to decide is empirically—try both (and Elastic Net) using cross-validation to tune the regularization strength (alpha/lambda) and compare validation performance, rather than deciding purely from theory.
what is diamond and circle shape in L1 and L2?	These are just pictures of the constraint region with two weights, w1 and w2. For L1 the penalty is |w1| plus |w2|, and keeping that under a limit traces a diamond, which has sharp corners sitting on the axes. For L2 it is w1 squared plus w2 squared, which traces a circle. Squaring gives the smooth circle and the absolute value gives the straight-edged diamond. The best solution usually touches the corner of the diamond, and at a corner one weight is exactly zero. A circle has no corners, so L2 shrinks but never zeroes.
How Dropout is helpful if it randomly zero out neurons during training?	Dropout helps because randomly turning off some neurons during training prevents the network from becoming too dependent on specific neurons or feature combinations. Each training step effectively uses a slightly different smaller network, so the model is forced to learn more robust and distributed representations. This reduces co-adaptation between neurons and helps prevent overfitting. During testing, dropout is turned off and all neurons are used, with the activations appropriately scaled.
Can we use one or multiple regularization techniques together ?	Yes. Combining L1 and L2 is called Elastic Net, and you get both feature selection and smooth shrinkage together.
does model forgets the last learning in drop out	No, nothing is forgotten. Dropout only switches neurons off temporarily during that one forward and backward pass. The weights themselves are kept safe and are never erased. In the next batch a different random set is switched off, and the earlier neurons come back with their learning intact. At test time all neurons are used together.

what is keras?	Keras is a library in Python. It lets you build and train neural networks with very few lines of code, so you write model.add for layers and model.fit to train instead of coding the maths yourself.
Is there a specific scenario when we should Dropout? or for all the scenarios?	Not for every case. Use dropout when the network is large, the data is limited, and you can see overfitting, meaning training loss keeps falling while validation loss rises. Skip it when the model is small or already underfitting, since it will only hurt. Also note it is applied in the hidden layers, never on the output layer, and it is switched off automatically at test time.
how are zeroing out neurons vs making weights zero different	Dropout zeroes the neuron's output for one batch only, and it is random and temporary. The weights stay untouched and the neuron is active again in the next batch. L1 makes a weight permanently zero as part of the learned solution. That connection is removed for good, which is why L1 is used for feature selection while dropout is only a training-time trick.
Which one is better technique Early stop and dropping?	Neither is universally "better". Early stopping (halting training once validation error stops improving) is simple and cheap but only prevents overfitting from too many training iterations, while dropout (randomly deactivating neurons during training) is more powerful for reducing co-adaptation and overfitting in larger neural networks; in practice they are often used together rather than as competing alternatives.
When should we use Ridge and when we should use Lasso in real-world scenarios?	Use Lasso when you have many features and suspect only a few actually matter, since it drives the useless ones to exactly zero and gives you a simpler, easier to explain model. Use Ridge when you believe most features contribute a little, or when features are strongly correlated, since it keeps them all and just shrinks them evenly. If you are unsure, Elastic Net combines both and is a safe default.
so how does the regularization process selection works, lets say I want to emphasis more on early stopping, can I explicitly select that before the training starts?	Yes. You can explicitly choose early stopping before training by setting the validation metric to monitor and a patience value. During training, if that metric stops improving for the specified number of epochs, training stops automatically. Similarly, L1/L2, dropout, and other regularization methods are usually configured beforehand and tuned using validation performance.

Prof mentioned a point about when Learning is said to complete. I kind of missed that. Is it the stage when training error and validation error become equal?	Not necessarily. Learning is considered complete when further training no longer improves the model's generalization, typically when the validation loss stops decreasing or starts increasing while training loss may still decrease. Training and validation errors do not need to become equal; a small gap between them is normal. If validation performance has stabilized, that is usually a good indication that training should stop.
How does dropout helps normalization	Small clarification, dropout helps regularization, not normalization. Those are two different things. Normalization rescales your input features so they are on a similar range. Dropout helps regularization by randomly switching neurons off, so the network cannot depend too heavily on any one neuron. It is forced to spread the learning across many, which reduces overfitting and improves generalisation.
How is removing the validation set even a maybe solution for the mentimeter question? Coz it's only hiding the problem. Without the validation set, there is no way to measure generalization	You are right. It does not fix overfitting at all, it only removes your ability to see it.
whta are epochs	An epoch means the model has seen the entire training dataset once. For example, if you have 1,000 training examples, then after the model has processed all 1,000 examples once, that is 1 epoch, training for 10 epochs means it sees the full dataset 10 times.
K in K means clustering is a hyperparameter then?	yes...k is number of clusters
For setting up the hyperparameter for the first time How to set it ? Do we need to understand the data and business ?	Yes, both help, but you do not start from scratch. Begin with known safe defaults, such as learning rate 0.001 with Adam, batch size 32, and dropout 0.2 to 0.5. Data size guides the model size, small data means a smaller network. Business need guides things like the decision threshold and which errors matter more. Then tune from there using the validation score.
What is a "Bigger" model?	more number of parameters... eg number of layers
what if we want to change hyperparameters after validation results?	Then we simply adjust the hyperparameters based on the validation performance and train the model again. For example, if validation loss is high due to overfitting, we might increase regularisation, increase dropout, reduce model size, or use early stopping, the test set should still remain untouched until the final evaluation.
Apply L2 Regularization (i.e Weigth Decay) in case of model overfitting..This should be handled by backpropagation Internally..	With L2 regularisation, the penalty term is added to the loss, and during backpropagation its gradient is automatically included, so the optimizer updates the weights while also shrinking large weights.

does shallow network learns any feature on its own?	Yes. A shallow network can still learn features automatically from data through its learned weights, but the features are usually simpler because there are fewer layers. A deep network can build features hierarchically, for example edges → shapes → object parts → objects, while a shallow network has less capacity to learn such complex representations.
what is alpha	Alpha is the regularization strength, the same thing written as lambda in the formula. It controls how hard the penalty pushes the weights down. A small alpha means a weak penalty, so the model stays complex and may overfit. A large alpha means a strong penalty, weights shrink a lot and the model may underfit. You tune it using validation performance.
Are there more Models like Neuwral Networks ?	Yes. Neural networks are only one family of machine-learning models, others include linear regression, logistic regression, decision trees, random forests, SVMs, k-nearest neighbours, and gradient-boosting models such as XGBoost. Within neural networks themselves, there are many architectures such as CNNs, RNNs/LSTMs, and Transformers.
If the training R^2 is high but the test and cross-validation R^2 scores are low, how can we determine whether the problem is the polynomial degree or insufficient Ridge regularization?	Test it directly. Keep the degree fixed and sweep alpha across a wide range. If a higher alpha closes the gap and lifts the test score, regularization was the issue. If even strong alpha does not help, or the model then starts underfitting, the degree is too high for your data, so reduce the degree. Plotting validation R2 against alpha, and separately against degree, makes this obvious.
How many number of dimensions neural network can support ?	There is no fixed limit in the maths. A network can take thousands or even millions of input features, an image of 224 by 224 pixels is already about 150,000 inputs. The real limits are practical limitations, your GPU memory and the amount of data you have. With very many features and little data you will overfit, so people reduce dimensions or add regularization.
in backpropagation, Are weights for only first layer of neurons updated or is it updated for neurons of all layers?	Every layer is updated. Backprop starts at the output, computes the gradient there, then passes the error backward through each layer using the chain rule until it reaches the first layer. So in one pass you get a gradient for every weight in the network, and gradient descent updates all of them together.

what is graph neural network ? how does it work?	A graph neural network works on data shaped as a graph, meaning nodes connected by edges, such as a social network, a molecule, or a payment network. Each node starts with its own feature vector. In every layer, a node collects the vectors of its neighbours, combines them, and updates its own vector. After a few layers each node carries information about its wider neighbourhood, and you use those vectors to predict node labels, links, or a property of the whole graph.
ANy LLM model is developed in India?	Sarvam AI, Kutrim, BharatGPT, SUTRA
which type of ML technique used by modern LLMs like chatgpt, Claude ?	They are deep learning models built on the transformer architecture, which uses self attention to relate every word to every other word in the text. We would be looking at Transformers model soon in this course.
Is the model predicted value guaranteed to converge towards the true label via backprop ?	No. Backpropagation computes gradients that tell the optimizer how to reduce the loss, but it does not guarantee that predictions will reach the exact true label. Convergence depends on factors such as the model architecture, data quality, learning rate, loss function, optimizer, and whether the problem itself is learnable.
is bias in bias-variance tradeoff different from bias i.e. y intercept at x = 0 in linear equation?	In linear regression, the bias/intercept b in y = wx + b is a trainable model parameter that shifts the prediction up or down. In the bias-variance trade-off, bias means systematic prediction error, namely how far the model's average prediction is from the true relationship.
Are the real Models hybrid of diffrent types of Models ?	Yes. Real systems mix architectures for what each does best. A vision language model uses a CNN or vision transformer for the image and a transformer for the text. Speech systems pair an audio encoder with a language model.
can it be matematically proved that variance is high in overfitting?	Yes. The expected test error decomposes exactly into three parts: bias squared, variance, and irreducible noise. This is a proven result, not just an observation.
how many times we can use the validation data set. If used multiple times, does it have a downside?	You can use it many times, that is exactly what it is for, tuning and comparing models. The downside is that with each decision you make based on it, you slowly fit your choices to that particular set. Its score then looks better than reality. This is why the test set is kept separate and touched only once. If you are tuning heavily, use cross validation or refresh the validation split.
cant we use gradient descent as loss in NN	No. Loss function tells us what quantity should be minimised, while gradient descent is the optimisation method used to minimise that loss. For example: MSE = loss function, gradient descent = method that changes weights to reduce MSE.

Can we replace any non linear function witha combination of linear function and hence solve any pattern with neural network	Not with linear functions alone. Stacking linear layers only gives you another linear function, however many you add, so the network could never bend. What makes it work is linear layers plus a nonlinear activation like ReLU. With that combination the universal approximation theorem says a network can approximate almost any continuous function to the accuracy you want, given enough neurons.
what is the diff between cross validation and train/val/test split	In a train/validation/test split, the dataset is divided once: train for learning weights, validation for tuning hyperparameters, and test for final evaluation. In cross-validation, we repeatedly rotate which part acts as validation, for example 5-fold CV trains 5 times using a different fold for validation each time, giving a more stable estimate of model performance.
will slow learning rate require high number of epochs to reach convergence?	Yes. A smaller learning rate means each weight update is smaller, so the model usually needs more epochs or training steps to reach convergence. The real benefit is more stable learning, if the learning rate is too large, the model may overshoot the minimum instead of converging.
thank you prof Anurup Naskar, in this case which is better. we have option to rotate using cross validation, why not that can be defualted for model may be my question is wrong, please clarify	Hello Sir, your question is actually valid. Cross-validation gives a more reliable estimate because every fold gets a chance to be validation data, but it requires training the model multiple times, so for large neural networks it can be very expensive. Therefore, a fixed train/validation/test split is often preferred for deep learning, while cross-validation is more common when the dataset is relatively small.
In which type of problems we use Ridge and Lasso?	Ridge and Lasso are mainly used in regression problems when we want to reduce overfitting. Ridge is useful when most features are relevant but their coefficients should be kept small, especially when features are highly correlated. Lasso is useful when there are many features and we expect only a few to be important, because it can shrink some coefficients exactly to zero and effectively perform feature selection.
so Gradient descent will eventually optimize/adjust weights	yes
what is an MLP	An MLP or Multi-Layer Perceptron is a basic neural network made of an input layer, one or more hidden layers, and an output layer. Each layer applies weighted sums and activation functions, allowing the model to learn non-linear patterns from data.
how do we decide how many layer (unit) is needed in CNN?	There is no fixed rule for how many CNN layers or units to use, it is a hyperparameter chosen experimentally using validation performance. Start with a smaller network, increase depth/filters if it underfits, and reduce complexity or add regularisation if it overfits.
is convolution a linear function	Yes, convolution itself is a linear operation. It is just a weighted sum of the pixel values under the filter, so it obeys the linearity rules.

Is any other activation function can be used in place of Relu in CNN?	Yes, several. Leaky ReLU and ELU fix the problem of neurons getting stuck at zero. GELU and Swish are smoother and are common in modern architectures. Sigmoid and tanh can technically be used but are avoided in deep CNNs because their gradients vanish. ReLU remains the default since it is fast and works well, but the others are worth trying.
how are the filters selected in CNN?	In CNNs, the filters are usually not manually selected. They start with small random values, and during training the network automatically adjusts them using backpropagation and gradient descent. Over time, some filters learn to detect simple patterns like edges or colors, while deeper layers learn more complex patterns like textures, shapes, or parts of objects. So, we mainly choose things like the number and size of filters, while the actual filter values are learned from the data.
this filter is a hyperparameter? how are the values chosen?	In CNNs, the filters are usually not manually selected. They start with small random values, and during training the network automatically adjusts them using backpropagation and gradient descent. Over time, some filters learn to detect simple patterns like edges or colors, while deeper layers learn more complex patterns like textures, shapes, or parts of objects. So, we mainly choose things like the number and size of filters, while the actual filter values are learned from the data.
how is the filter or kernal determined?	The numbers inside the filter are learned, not chosen. They start as small random values, and backpropagation computes a gradient for each one, so gradient descent gradually adjusts them to whatever reduces the loss. What you do choose beforehand are the settings around it, the filter size such as 3 by 3, how many filters per layer, the stride and the padding.
do we have specific filters for each feature type in image like edge, background etc?	In classical image processing yes, people hand designed filters like Sobel for edges or Gaussian for blurring. In a CNN you do not design them. The filter values are weights, and the network learns them from the data through backpropagation. What emerges is that early layers learn edges and colours, middle layers learn textures and shapes, and deeper layers learn object parts.
How do we design filter table?	If you mean the filter/kernel values in a CNN, we usually do not design the table manually. We choose the filter size, such as 3×3 or 5×5, and the number of filters. The values inside each filter are initialized randomly and then learned automatically during training through backpropagation. In traditional image processing, we can manually design filters such as edge-detection or sharpening kernels, but in CNNs the network learns the most useful filters from the data.

how to select the values of filter	You do not select them. They are initialised randomly, usually with a scheme like He or Xavier initialisation, and then learned automatically during training through backpropagation. The only thing you set is the shape, such as 3 by 3 or 5 by 5, and how many filters the layer should have. The actual numbers inside are the model's job to figure out. Mostly you stay with 3 by 3, since stacking two of them covers the same area as a 5 by 5 but with fewer parameters and an extra nonlinearity.
how we get values for filter?	They begin as small random numbers, then during training backpropagation calculates how much each filter value contributed to the error, and gradient descent nudges it in the direction that lowers the loss. Repeat this over many batches and the filter values settle into patterns that detect useful features.
here the filter is same but the image matrix is moved across co-ordinates. how the same filter works in this scenario?	That is the whole point, and it is called weight sharing. The same filter is reused at every position because a feature like a vertical edge looks the same wherever it appears in the image. So one filter learns one pattern, and sliding it checks for that pattern everywhere. This is why a CNN needs far fewer weights than a dense network, and why it can spot an object no matter where it sits in the picture.
how is the filter values populated in CNN?	the filter values are learnable weights, we start with some random weights and update them through back prop
How does the Convulation matrix creates the filter?	The convolution does not create the filter, it is the other way round. The filter already exists as a set of weights in the layer, and convolution is just the operation of sliding it over the image and taking the dot product at each position. The output of that is the feature map.
in a CNN how does one come up with a filter? Are there specialised filters for detecting different shapes/ patterns?	You do not come up with it. You only declare how many filters and what size, and the values are learned from data by backpropagation. They do end up specialising, but that happens on its own. After training you find some filters responding to vertical edges, others to curves, colours or textures, and deeper ones to object parts. Nobody assigns those roles, they emerge because those patterns help reduce the loss. Look at responses to similar questions in Q&A for more clarity. Do let us know if there are doubt.

can you please explain how the current input image represents a vertical line?	if the middle row is 0, then the filter compares the pixels above and below the center. So it detects a strong top-to-bottom change, which corresponds to a horizontal edge, not a vertical edge. In simple terms: middle row 0 -> compare top vs bottom -> detect horizontal edges.
Is there a chance of neuron always producing negative values? In that case, how do these activation functions work as it will convert everything to 0?	Yes, that can happen, especially with ReLU. If a neuron keeps producing negative pre-activation values, ReLU outputs 0 every time, and its gradient is also 0, so the neuron may stop learning completely. This is called the dying ReLU problem. To reduce it, we can use better weight initialization, a smaller learning rate, or activation functions such as Leaky ReLU, which allows a small negative output and gradient instead of making everything exactly zero.
why matrix multiplication is done row to row instead of row to column in this example?	It is actually row-to-column, not row-to-row. In the slide, each row of W is dotted with the input column vector x, so each row produces one class score: score = W·x + b.
is the Filter/kernel dimention(3*3) variable?	Yes. The kernel size such as 3×3 is a hyperparameter and can be changed to 1×1, 5×5, 7×7, etc. Smaller kernels capture local details with fewer parameters, while larger kernels see a wider region at once but require more computation.
how are we deciding on the filter / kernel matrix?	In a CNN, we usually do not manually decide the exact numbers inside the filter/kernel matrix. We choose the kernel size (for example, 3×3), the number of filters, stride, and padding. The kernel values start with small random values and are then learned automatically through backpropagation. During training, the model adjusts these values so different filters become useful for detecting patterns such as edges, textures, shapes, and eventually more complex objects. Manually designed kernels are mainly used in traditional image processing, not in typical CNN training.
In this example how is the feture map size 3*3. shouldn't it be 2*2 matrix	For a 5×5 input with a 3×3 kernel, stride 1, and no padding, the output size is (5−3)/1 + 1 = 3, so the feature map is 3×3. It would become 2×2 only if the stride were 2.
In Feture map while calculating edges, How many edges we need to calculate ?	You do not decide a number of edges. You decide how many filters the layer has, and each filter produces one feature map. A layer with 32 filters gives 32 feature maps, each responding to a different pattern.
What if we pool min value in pooling ?	It exists but is rarely used. After ReLU most values are zero or positive, so min pooling would mostly return zeros and throw away exactly the strong activations you want to keep.
how to determine which filter to use? Any function/Algo we use based on image?	We usually do not manually choose the actual filter values, the CNN learns the filter weights automatically through backpropagation. We mainly choose hyperparameters such as kernel size and number of filters, then training learns which filters detect useful patterns like edges, textures, and shapes.

so here is the Filter the learning weights?	Yes, exactly. The filter values are the learnable weights of the convolution layer, just as the matrix W here is the learnable weight matrix of the fully connected layer. Both start random and both get updated by backpropagation. The only difference is that a filter is reused across every position of the image, while W is used just once on the flattened vector.
How we decide upon the values in Filter/kernel for matrix calculation in convolution?	At the start, the CNN usually initialises the kernel values automatically with small random numbers, for example a 3×3 filter might begin with random weights. During training, backpropagation + gradient descent repeatedly update those values so the filter becomes useful for detecting patterns such as edges or textures. So we choose the kernel size like 3×3, but the neural network learns the actual numbers inside it.
What should be the ideal filter table size? The input is 4*4 so can we pick 2*2?	Yes, on a 4 by 4 input a 2 by 2 or 3 by 3 filter is sensible. The rule is simply that the filter must be smaller than the input, and it should be small enough that the output feature map still has useful size. In practice 3 by 3 is the standard choice on real images. On a tiny 4 by 4 map, 2 by 2 works fine.
what is the logic behind deciding stride? always it is 1 or anyother value and why?	Stride is how many pixels the kernel moves each time, and it does not have to be 1. Stride = 1 preserves more spatial detail and gives a larger feature map, while stride = 2 or more downsamples the image, reducing computation but losing some detail. So we choose stride based on whether we want fine feature extraction or spatial reduction.
How does the Input Image got converted to 5X5 matrix?	The 5 by 5 was only a small example for the slide, so the arithmetic stays readable. A real image is not converted down to that. An image is already a matrix. Each pixel has a brightness value from 0 to 255, so a 224 by 224 colour image is a 224 by 224 by 3 grid of numbers, one layer each for red, green and blue. I hope that answers it. but if i misunderstood you question, please do ask here again.

shouldn’t the matrix multiplication work horizontal to vertical in the example	You are right about matrix multiplication, and that is exactly what the fully connected slide does, W is 3 by 4 and x is 4 by 1, so rows times columns gives 3 by 1. Convolution is different though. Despite the multiply sign on the slide, it is not matrix multiplication. It is element by element multiplication of the patch and the filter, position by position, and then all nine products are added to give one number.
For the filter how do we get to know that the weights we have arrived at are now appropriate?	We usually do not judge one filter separately :). The filter weights are considered appropriate when, after training, they help reduce the loss and the model performs well on the validation set. If training loss keeps improving but validation performance gets worse, the learned filters may be overfitting rather than becoming better.
What does the term “kernel” mean in this context?	In CNNs, a kernel is simply a small matrix of learnable weights, also called a filter, that slides across the image. At each position it combines with the local pixels to produce one value in the feature map, helping detect patterns such as edges, textures, and shapes.
does each convolution + RELU layer has its own Filter matrix?	Yes, and in fact each layer has many filters, not just one. A layer with 32 filters has 32 separate weight matrices, each learning a different pattern and each producing its own feature map. ReLU has no weights at all, it simply applies max of 0 and the value to whatever the convolution produced. So all the learning sits in the convolution filters.
so why can't we directly fine tune weights using GD	You do tune the weights using gradient descent, but gradient descent needs the gradient for each weight, and it cannot produce that by itself. Backpropagation is simply the efficient method for computing all those gradients in one backward sweep. Without it you would have to nudge each weight separately and re-run the whole network to measure the effect, which for millions of weights is impossible.
Is this necessary to keep the layers in particular order in CNN?	Yes, some of the order is essential. Convolution must come before its activation, otherwise stacked convolutions collapse into one linear operation. Pooling must come after convolution, since there is no feature map to pool otherwise. Flatten must come before the fully connected layer, and softmax must be last.
how does one know how many stacks are needed in CNN? is this something the model determines?	The model does not decide it. Depth is a hyperparameter you choose before training.

is CNN only for images ?	No, images are just the most common use. A CNN works on any data where nearby values are related. One dimensional CNNs are used on text, audio waveforms, ECG signals and time series. Three dimensional CNNs are used on video and medical scans like CT and MRI. The same idea applies throughout, a small filter slides over the data looking for local patterns.
What flatten layer do?	A flatten layer converts multi-dimensional feature maps into a single 1D vector so they can be passed to a fully connected (dense) layer. For example, if a CNN produces feature maps of size 7 × 7 × 64, flattening converts them into a vector of length 3136. It does not learn anything or change the values; it only changes the shape of the data.
why can't deep NN directly be used- why do need to use CNN?	You can, but it works badly on images. A dense layer on a 224 by 224 by 3 image would need about 150,000 weights per neuron, so the model becomes huge and overfits quickly. It also destroys structure. Flattening throws away which pixels were neighbours, and a cat shifted a few pixels looks like completely different input.
each filter gives a feature map which is a vector. then we have multiple feature maps. so what is the dimention of the input vector for one image?	Small correction first, a feature map is a 2D grid, not a vector. So a layer's output is a 3D block of height by width by number of filters. The flattening into a vector happens only once, just before the fully connected layer. If the final block is 7 by 7 by 512, the vector length is 7 times 7 times 512, which is 25088.
will the size of kernel impact prediction?	Yes, it does. A smaller kernel like 3 by 3 sees a narrow region and picks up fine detail, while a larger one like 7 by 7 sees a wider region and captures broader structure but costs many more parameters.
is `out` arg in con2d represents filter or the output feature map?	Both, because they are the same number. The out argument is the number of filters in that layer, and since each filter produces exactly one feature map, it is also the number of output feature maps. So Conv2d(1, 16, 3) means 16 filters, producing 16 feature maps, which then become the 16 input channels for the next layer.
how to decide i=on the number of filters	It is a hyperparameter you choose, not something learned. The usual pattern is to start small and double as you go deeper, for example 16, then 32, then 64, then 128.

is pooling a way to feature engineering ?	Not quite. Feature engineering is when you manually create features from raw data using your own domain knowledge, before training. Pooling is a fixed operation inside the network with no learnable weights. It just downsamples, keeping the strongest response in each window. So it reduces size and gives some tolerance to small shifts, but it does not create new features the way feature engineering does.
how many hidden layers in fully connected layer in CNN?	There is no fixed number. In a CNN, after convolution/pooling, the fully connected part can have one hidden layer or several hidden layers depending on model complexity. For simple CNNs, often 1–2 fully connected hidden layers are enough, deeper modern CNNs may even use just one final linear layer after global pooling.
pytorch vs tensorflow? are there any diff in these libraries to enable us for Deep learning models like CNN?	Both can build any deep learning model including CNNs, so capability is not the difference. PyTorch is more Pythonic and easier to debug, and it dominates research and most new work. TensorFlow with Keras is simpler for quick model building and has stronger production and mobile deployment tooling. Learn either one, the concepts transfer directly.
can i solve same use case with two model? for predicting future house price with supervised learning vs deep learning?	Yes. You can solve the same house-price prediction problem with both a traditional supervised learning model and a deep learning model. For example, you could compare Linear Regression or Random Forest against an MLP neural network using the same training data and then choose the model that performs better on the validation/test set.
what does maxpool do?	Max pooling slides a small window, usually 2 by 2, over the feature map and keeps only the largest value in each window. A 28 by 28 map becomes 14 by 14. 2 benefits. It cuts the size, so fewer computations and fewer parameters later. And keeping the strongest response means small shifts in the image do not change the output much.
Is there any limit till when this conventional+ReLu loop to continue? How the number is decided?	There is no fixed limit to how many Convolution → ReLU blocks a CNN can have. The number of such blocks is a hyperparameter, chosen based on validation performance, computational cost, and whether adding more layers actually improves the model. If extra layers no longer improve validation performance, or start causing overfitting, we usually stop increasing the depth.

Any example for CNN used other than image ?	Text classification uses a 1D CNN, where the filter slides across a few consecutive words to pick up phrases, which is how sentiment models often work. ECG signals for heartbeat abnormality detection, audio spectrograms for speech and sound classification, and sensor time series for machine failure prediction all use 1D CNNs on the same principle. Molecules and protein structures use CNNs for drug discovery. Satellite and radar data are used for weather prediction and crop monitoring. Video uses 3D CNNs for action recognition.
For a two-hidden-neuron network (input x=1.0, target y=1.0, learn rate η=0.5): • Hidden neuron 1: $z_1 = w_1 \times x = 0.6 \times 1.0 = 0.6 \rightarrow h_1 = \sigma(z_1) = 0.6457$ • Hidden neuron 2: $z_2 = w_2 \times x = 0.4 \times 1.0 = 0.4 \rightarrow h_2 = \sigma(z_2) = 0.5987$ • Output: $z_3 = w_3 \cdot h_1 + w_4 \cdot h_2 = 0.8 \times 0.6457 + 0.5 \times 0.5987 = 0.8159 \rightarrow \hat{y} = \sigma(z_3) = 0.6934$ • MSE Loss: $L = \frac{1}{2} \times (y - \hat{y})^2 = \frac{1}{2} \times (0.3066)^2 = 0.0470$ In this neural network example, why is z3z_3z3 calculated as w3h1+w4h2w_3h_1 + w_4h_2w3h1+w4h2 instead of using the same formula as z1z_1z1 and z2z_2z2?	It is actually the same formula, weighted sum of inputs. The difference is only what the inputs are at that point. For z1 and z2 the input is the single value x, so the sum has one term. For z3 the inputs are the two hidden outputs h1 and h2, so the sum has two terms, one weight for each incoming connection. More incoming connections simply means more terms.
if my image is colour/3d will it be having conv2d or conv3d? what decides conv dimension 2d ord 3d?	A normal colour image is still processed with Conv2D, because the convolution moves across two spatial dimensions: height and width. The RGB channels are treated as input channels, not as a third spatial dimension. Conv3D is used when the data genuinely has three spatial/ordered dimensions to slide across, such as height × width × depth in a CT/MRI volume, or sometimes time × height × width in video.
Instead of CNN, are transformers (Self-attention) better for text classification as it can tend to words which have dependencies quickly?	Yes, often Transformers are better suited for text classification because self-attention can directly relate words that are far apart in a sentence. A CNN mainly captures local patterns through its receptive field, while a Transformer can model long-range dependencies more directly, although CNNs can still work very well for simpler or smaller text-classification problems.
Why fix size image in CNN ?	The convolution and pooling layers actually handle any size. The problem is the fully connected layer, since it needs a fixed length input vector, and the flattened size depends on the image dimensions. That is why images are resized before training.
So the Flatten layer converts a multi-dimensional feature map into a single 1-dimensional vector ?	yes

Can the one Hot vector representation of a word be universal? irrespective of the current prob we are solving	Not really. A one-hot vector is defined relative to a particular vocabulary, so the same word may have a different vector position in a different dataset or model. For example, if “cat” is vocabulary item 3 it may be [0,0,1,0], but in another vocabulary it could be item 1, giving [1,0,0,0].
how are we feeding the vocabulary to the model?	You do not feed the vocabulary itself. You build it once from your training text by listing all unique words and giving each an index, and that mapping is saved. At runtime each word is converted to its index, and the model looks up the row for that index in the embedding matrix. So the model only ever receives numbers, and the vocabulary is just the lookup table you keep alongside it.
In this example for 8 diemnsional vector, can there be two 1's for one word?	No. In a true one-hot vector, exactly one position is 1 and all the others are 0. So for an 8-dimensional vocabulary, a word might be [0, 0, 1, 0, 0, 0, 0, 0]. If two positions are 1, that is no longer one-hot encoding.
if we have duplicate values in dimentional vector in one-Hot VEctor, what happen?	In a true one-hot vector there is only one 1, so duplicates cannot occur, each word has its own unique index. What you see on this slide is bag of words, where several positions are 1 because the sentence contains several words. That is intentional. If a word repeats in the sentence, that position becomes a count like 2 instead of 1, which is called a count vector.
What is the usecase when we should be using One-Hot Vector representation? Since with one-hot the space required will increase with the vocabulary.	One-hot vectors are mainly useful when the number of categories is small and we simply need to represent each category distinctly, without implying any relationship between them. For example, representing categories like red/green/blue or class labels can work well with one-hot encoding. For a large NLP vocabulary, however, one-hot vectors become extremely high-dimensional and sparse—for a vocabulary of 50,000 words, every word would require a 50,000-dimensional vector with only one 1. They also do not capture similarity between words. Therefore, modern NLP models usually use token IDs followed by an embedding layer, which converts each token into a much smaller dense vector that can also learn semantic relationships.
Why fully connected layer require only fixed length input?	Because a fully connected layer has a fixed weight matrix whose dimensions are decided when the model is created. If the layer expects 512 input values, it has weights shaped to multiply exactly those 512 values, so an input of 400 or 600 values would not match. That is why CNN feature maps are often flattened or pooled to a fixed size before entering the fully connected layer.

Can we not consider the vocabulary universal?	In principle yes, but it is impractical. A universal vocabulary would need every word of every language, plus names, slang, typos and new words, giving vectors millions of positions long and mostly zeros.
Is 1-hot encoding used only to train NLP based model?	No. One-hot encoding is used for any categorical variable, not just NLP. For example, categories such as colour = {red, green, blue}, city = {London, Delhi, Paris}, or product type can all be represented using one-hot vectors.
What ia anonymization and embeddings?	Two separate things. Anonymisation is removing or masking personal details from data, such as replacing names, phone numbers or account numbers, so an individual cannot be identified. It is a privacy step, not a modelling step. An embedding is a short dense vector of real numbers that represents a word, learned from data. Similar words get similar vectors, so king and queen sit close together, which is exactly what one-hot cannot do.
So if i have 2 sentences, we will only consider the unique list of words in those two for sentences to create one hot vector?	Yes. We first build a vocabulary containing the unique words across those two sentences, and the vocabulary size determines the length of every one-hot vector. For example, if the two sentences together contain 8 unique words, every word is represented by an 8-dimensional one-hot vector with exactly one 1.
Pls explain in detail for dense embedding?	A dense embedding is a fixed-length vector representation (typically continuous, real-valued, and low-dimensional relative to vocabulary/feature size, e.g., 128–1536 dimensions) where most or all values are non-zero, encoding semantic meaning such that similar items (words, sentences, images) end up close together in the vector space—unlike sparse representations (like one-hot or TF-IDF vectors) which are mostly zeros and capture no inherent similarity structure. These embeddings are typically learned by neural networks (e.g., Word2Vec, BERT, or sentence-transformer models) trained on large datasets so that geometric distance/similarity (cosine similarity, dot product) between vectors reflects real-world semantic or contextual relationships, making dense embeddings the standard representation used for tasks like semantic search, retrieval-augmented generation (RAG), and clustering.
How are these values in the matrix of dense embedding calculated	They are learned, not calculated by a formula. The embedding matrix starts as random numbers, one row per word.

is dense embedding also part of the flattened structure step we saw in the diagram?	Not exactly. Dense embeddings and flattening are different operations. An embedding layer converts each token ID into a dense vector containing learned features. So a sentence becomes a sequence (matrix) of dense vectors. Flattening, if used, then takes that multi-dimensional representation and converts it into one long 1D vector before passing it to a fully connected layer. So the flow could be: Token IDs -> Dense Embeddings -> Flatten -> Dense/Output layer. However, modern NLP models like RNNs and Transformers usually do not simply flatten all embeddings; they process the sequence structure directly.
How this Vector mappings are done? I mean car- 0.2, Kitten etc with some numeric value! Are these globally defined or complete local	They are local to the model that learned them. There is no global standard value for a word, and the numbers from one model mean nothing in another. You get them either by training on your own text, or by downloading pretrained vectors like Word2Vec, GloVe or fastText.
@ Sombit_Bose_TA - didnt understand. so the steps were conv-relu-pooling-conv-relu-flattened structure-mlp-output. where does dense embedding come in this flow?	For an image CNN, there is usually no embedding layer: Image -> Conv -> ReLU -> Pooling -> Flatten -> MLP -> Output. Dense embeddings are mainly for text, where token IDs are first converted into vectors: Text -> Token IDs -> Embedding -> Conv/RNN/Transformer.
Does skip-gram/CBOW also checks for grammar? Is this used for the LLM response construction in plain text?	Not explicitly. Skip-gram and CBOW only learn from which words appear near each other, so no grammar rules are coded in. LLMs do not use these for generating text. They use their own learned embeddings inside a transformer, which handles word order and grammar properly.
what is the difference if the model is taking Skipgram or CBOW?	They are the same idea run in opposite directions. CBOW takes the surrounding words and predicts the missing middle word. Skip-gram takes the middle word and predicts the surrounding ones. CBOW is faster and works better for frequent words. Skip-gram is slower but handles rare words and small datasets better, so it is usually the safer choice.
what is window in Word2Vec?	In Word2Vec, the window is the number of neighbouring words around a target word that we consider as its context. Example: in “the cat sat on the mat”, with window = 2 and target word “sat”, the context words are “the”, “cat”, “on”, and “the”.

How are fasttext and GloVe different from word2vec	Word2Vec learns from local context windows only, predicting nearby words. GloVe instead uses global co-occurrence counts across the whole corpus, factorising that matrix, so it captures corpus-wide statistics. fastText is Word2Vec extended to subword pieces. A word is represented by its character n-grams, so it can build a vector for a word it never saw in training, and it handles spelling variations and morphologically rich languages much better.
then what is relevance for word2vec while fasttext and GloVe has come? or can we use all of them in unison?	They came around the same time and none replaced the others, they trade off differently. Word2Vec is still relevant because it is fast, simple and a strong baseline, and it is the clearest way to understand the concept. You would normally pick one, not combine them, though people sometimes concatenate vectors from two sources. In practice today, if context matters, most work has moved to BERT-style contextual embeddings anyway.
Sir, how does BERT distinguish between ‘financial bank’ and ‘river bank’ when Word2Vec creates only one vector for both meanings?	Word2Vec gives one fixed vector for “bank,” regardless of the sentence. BERT creates contextual embeddings, meaning the vector for “bank” changes based on the surrounding words. In “I deposited money in the bank,” attention to words like money and deposited makes BERT represent bank as a financial institution, while in “I sat on the river bank,” words like river and sat make it represent the river-side meaning. So, BERT uses context through self-attention to disambiguate word meanings.
While designing a model for a specific use case, what are the aspects that need to be taken into consideration? Can one model be used for multiple use cases or it is always better to have one model for one use case	Key aspects are the data type and volume, the output you need, accuracy versus latency, cost and hardware, interpretability if it is a regulated domain, and how often the data will drift. Both approaches are valid. One shared backbone with separate heads works well when tasks are related, since they reinforce each other. Separate models are better when tasks are unrelated or have very different accuracy and latency needs, and they are easier to debug and update independently.
How vector will understand what bank is referred to like river bank or financial bank or any other bank etc	A static vector cannot, which is exactly the point of this slide. Word2Vec gives bank one fixed vector, averaging all its meanings. Contextual models like BERT solve it. They read the whole sentence and compute a fresh vector for bank each time, so the surrounding words money or river pull it towards the right meaning. Same word, different vector, depending on the sentence.

In CNN, can convolution be mentioned as correlation as well, since it looks to be a similar concept?	You are right, and this is a well known point. What deep learning frameworks call convolution is technically cross-correlation. True mathematical convolution flips the kernel first, and libraries skip that flip. It makes no difference in practice, because the filter values are learned anyway, so the network simply learns the flipped version if needed. The name stuck for historical reasons.
Does Matrix pooing works same way for grey and coloured images?	The only difference is how many channels there are. A grayscale image has 1 channel, colour has 3. But after the first convolution layer both become a stack of feature maps anyway, so from that point pooling treats them exactly the same.
I believe a large portion of the use cases (where we need determinsitic output) belong to ML, whereas a smaller portion related to undeterminstic output, or generative output belong to Foundational Model+Generative AI. What's the reason Gen AI is getting some much attention/hype compared to ML	Your read is fair, most business value still sits in classical ML for prediction and scoring. Generative AI draws attention because it is visible and general purpose. One model handles many tasks without task-specific training, the interface is plain language so non-technical users can use it directly, and it opened up unstructured text, image and voice work that was previously hard. Both coexist, ML for deterministic decisions, generative AI for language and content.
What are key differences between PyTorch and Tensor Flow? Which one is better? Also, what is Keras?	PyTorch and TensorFlow are both popular deep-learning frameworks used to build and train neural networks. PyTorch is generally considered more Python-friendly and intuitive, so it is widely used in research and experimentation. TensorFlow has traditionally been strong for production deployment and comes with a broad ecosystem for serving, mobile, and large-scale training. Today, both are capable of research and production, so “better” depends on your use case; for learning and research, PyTorch is often easier to start with, while TensorFlow can be attractive if you are already using its ecosystem. Keras is a high-level neural-network API that makes model building easier by hiding much of the low-level complexity. It is commonly used with TensorFlow
What is CUDA? How it is related to GPUs?	CUDA is NVIDIAs software platform that lets programs run general computations on a GPU instead of only graphics. Note it works only on NVIDIA hardware.

Why unsqueeze is done?	It adds an extra dimension of size 1, because PyTorch layers always expect a batch dimension. If you have a single image of shape 1 by 28 by 28, the model actually wants 1 by 1 by 28 by 28, meaning batch of one. So you call unsqueeze at position 0 to add it. Without that you get a shape mismatch error.
What is the use of squeeze	It adds an extra dimension of size 1, because PyTorch layers always expect a batch dimension. If you have a single image of shape 1 by 28 by 28, the model actually wants 1 by 1 by 28 by 28, meaning batch of one. So you call unsqueeze at position 0 to add it. Without that you get a shape mismatch error.
what is unsqueeze(2)?	It inserts a new dimension of size 1 at position 2, counting from zero.
How do we convert the audio values ?	In case of audio values you need a diff encoding strategy. Common audio encoding techniques depend on what we want to represent. Raw audio can be represented as a waveform, where we directly use amplitude values over time. We can also convert audio into frequency-based representations such as a spectrogram, which shows how frequencies change over time, or a Mel-spectrogram, which uses a frequency scale closer to human hearing and is very common in speech/audio deep learning. Another popular representation is MFCC (Mel-Frequency Cepstral Coefficients), which gives compact features useful for speech recognition and speaker-related tasks. Modern models may also learn audio embeddings directly from raw audio or spectrograms using neural networks such as CNNs, Transformers, or models like wav2vec.
suppose original tensor is 1 d like [1,2,3]. if i pass unsqueeze(2,1), will it add 2 columns ?	No. unsqueeze takes only one argument, the position, so unsqueeze(2, 1) would give an error. And it never adds columns of data, it only adds a dimension of size 1. On [1, 2, 3], which has shape (3), unsqueeze(0) gives shape 1 by 3 and unsqueeze(1) gives 3 by 1. Position 2 is out of range here. Before: x = [1, 2, 3], shape (3) After x.unsqueeze(0): [[1, 2, 3]], shape (1, 3), one row of three values. After x.unsqueeze(1): [[1], [2], [3]], shape (3, 1), three rows of one value each. The numbers are unchanged in both cases, only the brackets and the shape differ.

Difference between reshape and view. can you explain?	Both change the shape without changing the data. The difference is what they need underneath. view requires the tensor to be contiguous in memory and returns a view sharing the same storage, so it fails after operations like transpose. reshape tries view first, and if that is not possible it silently makes a copy. So reshape is safer and rarely fails, view is stricter but guarantees no copy. x = torch.arange(6) gives [0,1,2,3,4,5], shape (6). Both x.view(2,3) and x.reshape(2,3) work and give the same result. Now y = x.view(2,3).t(), which transposes to shape (3,2) and is no longer contiguous. Calling y.view(6) throws an error, but y.reshape(6) works because it quietly makes a copy.
what is advantage of multiple epochs if it runs on same data set? is it shuffled on every next epoch?	One pass is not enough because each batch only nudges the weights a little. Multiple epochs let the model keep refining and gradually converge to a lower loss. And yes, the data is usually shuffled every epoch, which you set with shuffle equals True in the DataLoader. That changes the batch composition each time, so the model does not learn the order itself and the gradients are less correlated.
One Epoch is covering one cycle of Forward Propogation + Back Propogation?	Almost, but that describes one iteration, not one epoch. A forward pass plus a backward pass plus a weight update happens once per batch. An epoch is one full pass over the entire training set, so it contains many such iterations. With 10,000 samples and a batch size of 100, one epoch is 100 forward and backward cycles.
Do we need to handcode the backpropagaton in pytorch? can't it be put it into callable function/object where i mention the architecture and the traininig strategy (something like h2o or Keras)?	No hand coding at all. PyTorch computes gradients automatically with autograd, so you only call loss.backward() and optimizer.step(). If you want the higher level style you describe, use PyTorch Lightning or fastai, where you declare the architecture and training strategy and the loop is handled for you. Keras on TensorFlow gives the same with model.compile and model.fit.
why not just do relu directly? it will cover both linear and not linear as well?	ReLU by itself has no weights, so it cannot learn anything. It is a fixed rule, output the value if positive else zero, with nothing to train. The convolution or linear layer is what holds the learnable weights and does the actual combining of inputs. ReLU only bends the result afterwards. You need both, weights to learn and nonlinearity to allow curves.